

7교시: logs/exec/inspect 기반 장애 분석

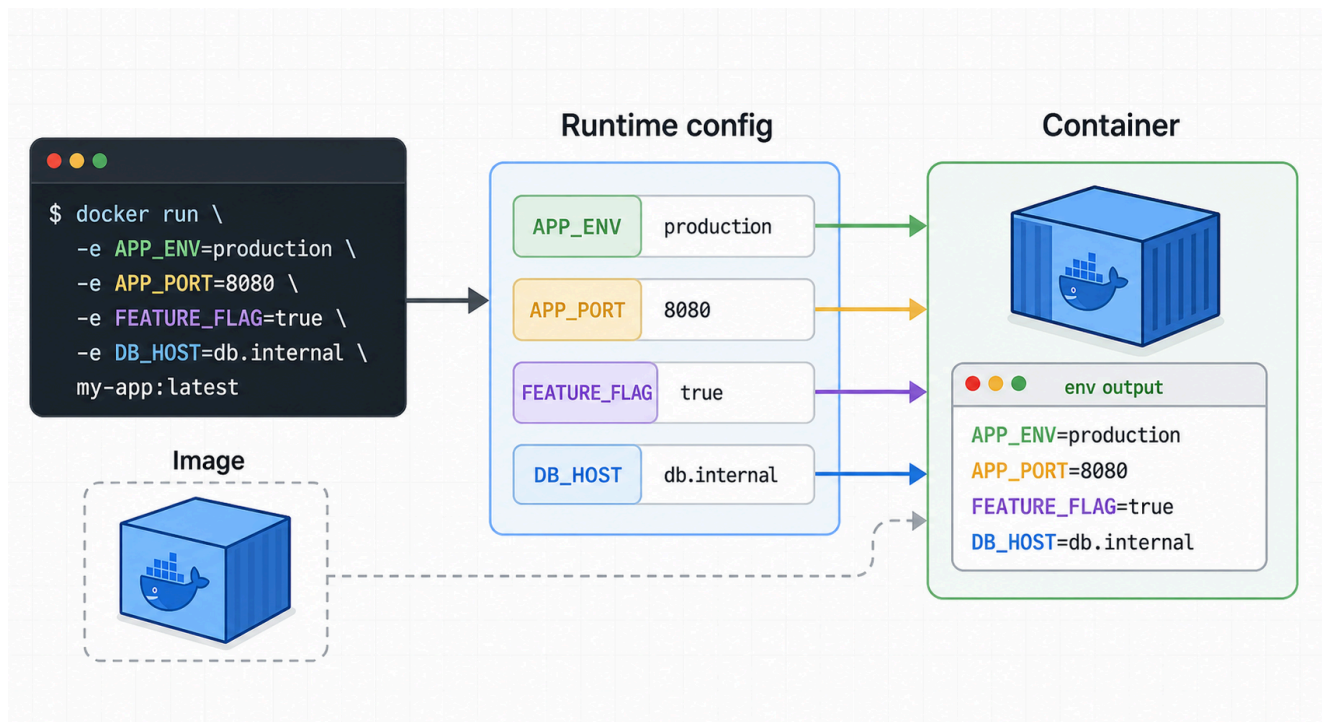
수업 목표

- runtime 장애를 port, env, network, volume, process/log 문제로 분류한다.
- docker logs, docker exec, docker inspect를 목적별로 사용한다.
- missing env, wrong port, stale volume을 RCA 형식으로 기록한다.

50분 흐름

시간	활동	비중	산출
0-8분	runtime failure taxonomy	설명 15%	failure map
8-20분	missing env failure	실행 25%	error evidence
20-30분	wrong port drill	실행 20%	port RCA
30-40분	inspect/logs/exec 분리	실행 20%	command map
40-50분	RCA note 작성	실행 20%	failure note

Visual 1: Runtime troubleshooting



환경변수, port, network, volume은 모두 runtime 조건이다. 장애 분석은 어떤 조건이 빠졌는지 evidence로 좁히는 과정이다.

핵심 설명

Day 3 장애는 대부분 image build 문제가 아니다. 같은 image라도 runtime 조건을 잘못 주면 container는 실패하거나, 실행되어도 기대한 방식으로 접근되지 않는다.

장애를 보면 먼저 build-time 문제인지 runtime 문제인지 나눈다. Day 3에서는 runtime 문제 안에서 port, env, network, volume, process/log 중 하나로 분류한다.

docker logs는 process가 출력한 메시지를 본다. docker exec는 실행 중 container 내부에서 명령을 실행한다. docker inspect는 Docker가 알고 있는 container metadata, mount, network, env, port mapping을 본다.

failure taxonomy

분류	대표 증상	첫 명령
Port	curl localhost 실패	docker ps PORTS
Env	DB 초기화 실패	docker logs
Network	service name 접근 실패	network inspect, same network test
Volume	데이터가 남거나 초기화 안 됨	docker volume ls, inspect
Process	container가 바로 종료	docker ps -a, logs

Drill 1: missing env

```
docker run --name paperclip-day3-postgres-missing-env postgres:16-alpine
```

Linux 사전 테스트 결과:

```
Error: Database is uninitialized and superuser password is not specified.
You must specify POSTGRES_PASSWORD to a non-empty value for the superuser.
```

해석:

- image pull/build 문제가 아니다.
- PostgreSQL official image의 runtime contract가 충족되지 않았다.
- -e POSTGRES_PASSWORD=...가 필요하다.

정리:

```
docker rm paperclip-day3-postgres-missing-env
```

Drill 2: wrong port

정상 web container를 -p 18083:80으로 실행한 뒤 잘못된 port로 접근한다.

```
curl -I http://localhost:80
curl -I http://localhost:18083
docker ps --filter name=paperclip-day3-web
```

해석:

- localhost:80 실패는 container 내부 80번이 달렸다는 뜻이 아닐 수 있다.
- docker ps에서 host port가 18083인지 확인한다.
- browser/curl은 host port로 접근한다.

Drill 3: stale volume

PostgreSQL container를 삭제하고 같은 volume을 다시 붙이면 기존 data directory가 유지된다.

```
docker rm -f paperclip-day3-postgres
docker run -d \
  --name paperclip-day3-postgres \
  -e POSTGRES_PASSWORD=paperclip \
  -e POSTGRES_DB=paperclip \
  -v paperclip-day3-pgdata:/var/lib/postgresql/data \
  postgres:16-alpine
```

해석:

- volume이 남아 있으면 DB 초기화가 새로 일어나지 않을 수 있다.
- env 값을 바꿨는데 DB 이름이나 초기 상태가 달라지지 않으면 stale volume을 의심한다.
- 새 실습을 원하면 새 volume name을 쓰거나 기존 volume을 삭제한다.

명령별 역할

명령	보는 것	적합한 질문
<code>docker ps</code>	running container, ports	떠 있는가, 어느 port인가
<code>docker ps -a</code>	종료된 container 포함	바로 죽었는가
<code>docker logs</code>	process output	왜 실패했는가
<code>docker exec</code>	내부 명령 실행	파일/DB readiness가 어떤가
<code>docker inspect</code>	metadata	mount/network/env가 어떻게 붙었는가
<code>docker volume ls</code>	volume object	데이터가 남아 있는가
<code>docker network inspect</code>	network membership	같은 network인가

좋은 RCA와 나쁜 RCA

나쁜 기록	좋은 기록
DB가 안 됨	POSTGRES_PASSWORD 없이 실행해 초기화 실패
포트가 이상함	-p 18083:80인데 localhost:80으로 확인함
Docker 문제	container는 Up이고 HTTP만 실패하므로 port mapping 확인 필요
데이터가 이상함	기존 paperclip-day3-pgdata volume을 재사용함
로그가 길다	readiness 줄과 error 줄만 발췌

핵심 유의사항

장애 분석은 명령을 많이 치는 것이 아니라 질문을 좁히는 과정이다. "어디가 문제인가"를 모르면 로그도 읽기 어렵다.

실패 로그는 과도하게 길게 붙이지 않는다. 핵심 error line, 실행한 명령, 정상 기준, 재확인 결과를 함께 기록한다.

`docker exec`는 내부를 고치는 도구로 쓰지 않는다. 초급 단계에서는 확인 도구로 사용한다. container 내부에서 수동 수정하면 image, volume, writable layer 경계가 흐려진다.

RCA 템플릿

```
## Runtime Failure RCA
```

- 증상:
- 실행한 명령:
- 정상 기준:

- 분류: port/env/network/volume/process
- 확인한 evidence:
- 핵심 error line:
- 조치:
- 재검증 명령:
- 재검증 결과:

마무리 점검

container가 바로 종료되면 먼저 ____를 본다.
 HTTP 실패는 먼저 ____에서 host port를 확인한다.
 DB 초기화 실패는 ____ 누락을 의심한다.
 데이터가 예상과 다르면 ____ 재사용 여부를 확인한다.

평가 기준

기준	2점 evidence
분류	runtime failure를 한 축으로 분류했다
logs	핵심 error line을 찾았다
inspect	port/mount/network 중 하나를 확인했다
RCA	재검증 결과까지 기록했다

공식 근거 링크

- Docker logs: <https://docs.docker.com/reference/cli/docker/container/logs/>
- Docker exec: <https://docs.docker.com/reference/cli/docker/container/exec/>
- Docker inspect: <https://docs.docker.com/reference/cli/docker/inspect/>